

```

require('dotenv').config();
const express = require('express');
const fetch = require('node-fetch');
const fs = require('fs');
const path = require('path');

const app = express();
app.use(express.static(path.join(__dirname, 'public')));

const SIGNALS_FILE = path.join(__dirname, 'signals.json');
const JOURNAL_FILE = path.join(__dirname, 'journal.json');
const TWELVEDATA_API_KEY = process.env.TWELVEDATA_API_KEY;
const OPENROUTER_API_KEY = process.env.OPENROUTER_API_KEY;
const MODEL = 'openrouter/free';

const TIMEFRAMES = { '1D': '1day', '4H': '4h', '1H': '1h', '15': '15min' };

function readJSON(file, fallback) {
  try {
    return JSON.parse(fs.readFileSync(file, 'utf8'));
  } catch (e) {
    return fallback;
  }
}

function writeJSON(file, obj) {
  fs.writeFileSync(file, JSON.stringify(obj, null, 2));
}

async function fetchCandles(symbol, interval) {
  const url = 'https://api.twelvedata.com/time_series?symbol=' + encodeURIComponent(
    symbol + '&interval=' + interval + '&outputsize=40&apikey=' + TWELVEDATA_API_KEY);
  const resp = await fetch(url);
  const data = await resp.json();
  if (data.status === 'error' || !data.values) {
    throw new Error('Twelve Data error for ' + symbol + ' ' + interval + ': ' + (
    data.error || ''));
  }
  return data.values.reverse().map(function (v) {
    return {
      time: v.datetime,
      open: parseFloat(v.open),
      high: parseFloat(v.high),
      low: parseFloat(v.low),
      close: parseFloat(v.close)
    };
  });
}

```

```

    };
  });
}

```

```

const SYSTEM_PROMPT = [
  'ROLE',
  'You are an institutional-level Price Action and Supply & Demand analyst follow',
  'Your only objective is to analyze the market exactly according to the rules be',
  'Never use opinions or predictions. Only analyze what price is currently doing.',
  '',
  'STRICT RULES',
  'Never use: RSI, MACD, EMA, SMA, Bollinger Bands, Fibonacci, VWAP, Volume Profi',
  'Use ONLY: Price Action, Supply & Demand, Market Structure.',
  '',
  'You will receive OHLC candle data (rolling window, most recent last) for Daily',
  '',
  'ANALYSIS ORDER (always in this exact order)',
  '1. Daily: Trend, Market Structure, Supply Zones, Demand Zones, Fresh Zones, Or',
  '2. 4H: repeat the same process.',
  '3. 1H: repeat the same process.',
  '4. 15M: repeat the same process.',
  '',
  'MARKET STRUCTURE',
  'Identify HH, HL, LH, LL. Determine Uptrend, Downtrend, or Range. Detect BOS, C',
  '',
  'SUPPLY & DEMAND RULES',
  'For every zone identify: Fresh, Original, Tested, Valid, Invalid. Score every',
  '',
  'QUALITY OF THE ZONE',
  'Evaluate: Momentum, Departure, Base Quality, Number of Base Candles, Distance',
  '',
  'LIQUIDITY',
  'Check: Liquidity Grab, Equal High, Equal Low, Stop Hunt, Compression, Expansio',
  '',
  'ENTRY RULES',
  'Only enter if ALL conditions align. Determine: Entry, Stop Loss, Take Profit,',
  '',
  'RISK MANAGEMENT',
  'Maximum Risk = 1%. Minimum RR = 2. If RR < 2, the decision MUST be NO_TRADE.',
  '',
  'TRADE FILTERS - force NO_TRADE if ANY of these are true:',
  'Trend conflicts across timeframes; Weak zone; Zone retested many times; No mom',
  '',
  'SCORING',
  'Trend Alignment = 20 max, Market Structure = 20 max, Supply Demand Quality = 2',
  'If total score is below 80, the decision MUST be NO_TRADE.',
  '',

```

```

'Never guess. If data is insufficient for a criterion, say so and lower the sco
'',
'OUTPUT',
'Respond with ONLY valid JSON, no markdown fences, no preamble, matching exactl
'{',
'  "symbol": string,',
'  "generated_at": string,',
'  "daily_bias": string, "h4_bias": string, "h1_bias": string, "m15_bias": stri
'  "trend": string,',
'  "market_structure": string,',
'  "current_supply_zone": string,',
'  "current_demand_zone": string,',
'  "fresh_zone": string,',
'  "original_zone": string,',
'  "liquidity_status": string,',
'  "bos": string,',
'  "choch": string,',
'  "zone_quality": {"momentum": string, "departure": string, "base_quality": st
'  "trade_filters_triggered": string[],',
'  "entry": number, "stop_loss": number, "take_profit_1": number, "take_profit_
'  "risk_reward": number,',
'  "confidence_pct": number,',
'  "score_breakdown": {"trend_alignment": number, "market_structure": number, "
'  "trade_score": number,',
'  "decision": "BUY" or "SELL" or "NO_TRADE",',
'  "reasoning": string',
'}',
'Use null for entry/stop_loss/take_profit/risk_reward if decision is NO_TRADE a
].join('\n');

async function analyzeWithAI(symbol, candleData) {
  const userMessage = 'Symbol: ' + symbol + '\nCandle data (JSON, per timeframe,

const resp = await fetch('https://openrouter.ai/api/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ' + OPENROUTER_API_KEY,
    'HTTP-Referer': 'https://sd-bot.local',
    'X-Title': 'SD Analysis Bot'
  },
  body: JSON.stringify({
    model: MODEL,
    messages: [
      { role: 'system', content: SYSTEM_PROMPT },
      { role: 'user', content: userMessage }
    ]
  })
}

```

```

    })
  });

const data = await resp.json();

if (!resp.ok) {
  throw new Error('OpenRouter API error: ' + JSON.stringify(data));
}

if (!data.choices || !data.choices[0] || !data.choices[0].message) {
  throw new Error('Unexpected OpenRouter response: ' + JSON.stringify(data));
}

const text = data.choices[0].message.content;
const clean = text.replace(/```json/g, '').replace(/```/g, '').trim();

try {
  return JSON.parse(clean);
} catch (e) {
  throw new Error('Model did not return valid JSON. Raw response: ' + text.slice(0, 1000));
}
}

async function runAnalysis(symbol) {
  const candleData = {};
  const timeframeKeys = Object.keys(TIMEFRAMES);
  for (let i = 0; i < timeframeKeys.length; i++) {
    const label = timeframeKeys[i];
    const interval = TIMEFRAMES[label];
    candleData[label] = await fetchCandles(symbol, interval);
  }
  const result = await analyzeWithAI(symbol, candleData);

  const signals = readJSON(SIGNALS_FILE, {});
  signals[symbol] = result;
  writeJSON(SIGNALS_FILE, signals);

  const journal = readJSON(JOURNAL_FILE, []);
  journal.push({
    symbol: symbol,
    logged_at: new Date().toISOString(),
    decision: result.decision,
    trade_score: result.trade_score,
    confidence_pct: result.confidence_pct,
    trend: result.trend,
    entry: result.entry,
    stop_loss: result.stop_loss,
  });
}

```

```

        take_profit_1: result.take_profit_1,
        take_profit_2: result.take_profit_2,
        take_profit_3: result.take_profit_3,
        risk_reward: result.risk_reward,
        reasoning: result.reasoning
    });
    writeJSON(JOURNAL_FILE, journal);

    return result;
}

app.get('/', function (req, res) {
    res.send('SD Analysis Bot is running. Try /dashboard.html or /api/analyze/AAPL'
});

app.get('/api/analyze/:symbol', async function (req, res) {
    try {
        const symbol = decodeURIComponent(req.params.symbol);
        const result = await runAnalysis(symbol);
        res.json({ status: 'ok', result: result });
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: err.message });
    }
});

app.get('/api/signals', function (req, res) {
    res.json(readJSON(SIGNALS_FILE, {}));
});

app.get('/api/journal', function (req, res) {
    res.json(readJSON(JOURNAL_FILE, []));
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, function () {
    console.log('Server running on port ' + PORT);
});

```